FLASK POSTGRES / SUPABASE JINJA2 RENDER / RAILWAY

School Attendance & Accountability Management System

Full Design Plan & Implementation Strategy

A complete, end-to-end blueprint covering architecture, database schema, page-by-page design, logic flows, security, and a step-by-step build roadmap from local development through to production launch.

Prepared as a working build guide • Version 1.0 • August 2026

Table of Contents

1. Problem Statement & Goals 3
2. Success Metrics 3
3. Final Technology Stack 4
4. System Architecture 5
5. Roles & Permissions Matrix 6
6. Database Schema (Full Design) 7
7. Complete Sitemap / Page Map by Role 12
8. Page-by-Page Design Detail 13
9. Timetable Capture: Full Data-Entry Workflow 18
10. Core Logic Flows (Step-by-Step) 21
11. Security, Privacy & Compliance 26
12. Environments, Deployment & DevOps 27
13. Project Folder Structure 29
14. Implementation Roadmap (Phase by Phase) 30
15. Testing & QA Checklist 34
16. Pilot Rollout Plan 35
17. Appendix: Environment Variables & Config 36

Page numbers are approximate section markers within this document, not fixed print pagination.

1. Problem Statement & Goals

1.1 The Problem

Students miss classes without the school detecting it in time. Roll calls are inconsistent — taken rarely, delegated to class representatives, or skipped entirely, sometimes deliberately by teachers avoiding a lesson. Where roll calls do happen, they exist as loose paper records that are never aggregated, so no one — not the teacher, not the Director of Studies (DOS), not the parent — can see a clear pattern before it is too late. By the time exam results reveal the gap, the intervention window has closed. Parents are left asking for accountability the school cannot give, because the underlying data was never captured in a usable form.

1.2 The Root Causes Being Solved

Root Cause	How the System Addresses It
Roll call is optional / delegated to students	Roll call becomes a mandatory, timetable-linked, teacher-owned digital action
No visibility until it's too late	Real-time dashboards at lesson, day, week, month, and term level
No accountability for teachers who skip class	Automatic flagging of sessions with no submitted roll call, tied to a named teacher
Paperwork is unusable for analysis	All attendance is structured, timestamped, queryable data from the moment of entry
New S1/S2 students exploit anonymity	Every student is registered with a photo and unique ID, searchable instantly by class
Parents cannot get a clear answer	A dedicated student profile page shows full attendance history on demand

1.3 Primary Goals

1. Make roll call mandatory, fast, and tied directly to the official timetable.
2. Automatically detect and flag missed or unsubmitted roll calls, per teacher, per session.
3. Turn raw attendance into visual insight (charts, highlights) at every level of aggregation.
4. Make any student instantly identifiable and searchable, with a photo, for staff and for parent-facing lookups.
5. Build the data model so it scales cleanly from S1/S2 (fixed subjects) to S3–S6 (electives, streams) without a rebuild.

1.4 Non-Goals (Explicitly Out of Scope for Version 1)

- Parent self-service login/portal — Phase 1 uses in-person, staff-assisted lookup only.
- SMS/email auto-notifications to parents — flagged as Phase 2.
- Student self-login to view their own attendance — Phase 2.
- Grades/exam results integration — a future, separate module, though the schema will not block it.

2. Success Metrics

Metric	Target
Roll call submission rate (sessions with a submitted roll call)	> 95% within one term of launch
Time for DOS to identify a struggling student	Under 2 minutes via dashboard, down from "not tracked at all"
Time for staff to answer a parent's attendance query	Under 1 minute via class → student search
Teachers flagged for repeated missed roll calls per term	Visible list maintained; count should trend down term over term
Pilot classes (S1/S2, 2 streams) fully onboarded with photos	100% before wider rollout

3. Final Technology Stack

Layer	Choice	Why
Backend framework	Flask (Python)	Lightweight, full control, server-rendered pages, easy to reason about for a small team
Templating / Frontend	Jinja2 + HTML + CSS + vanilla JS	No separate frontend app, no CORS/JWT complexity, faster to build and debug
Database (dev)	SQLite (local) or a free Supabase dev project	Fast local iteration; SQLAlchemy abstracts most differences from Postgres
Database (production)	Supabase Postgres	Managed, free-tier-friendly, relational — required for attendance analytics/JOINs
File/photo storage	Supabase Storage (private bucket)	Bundled with the database project, signed URLs for privacy
ORM & migrations	SQLAlchemy + Alembic	Version-controlled schema changes, safe to replay dev → staging → prod
Auth	Flask-Login + Werkzeug password hashing (custom, not Supabase Auth)	Single source of truth for identity; avoids mixing two auth systems
Forms & CSRF	Flask-WTF	Built-in CSRF protection, server-side validation
Charts	Chart.js (client-side, fed by JSON endpoints)	Free, lightweight, good enough for bar/pie/line/heatmap-style views
Hosting — application	Render or Railway	Persistent server process, supports scheduled jobs (cron), simple deploys from GitHub
Hosting — static assets (optional)	Vercel or served directly by Flask	Only needed if a separate marketing/parent-facing static site is added later
Email (password resets)	Flask-Mail + Brevo (Sendinblue) free SMTP tier	Not tied to Supabase Auth, works independently
Backups	Scheduled `pg_dump` to private storage	Free tier has no automatic backups — this is a self-managed safety net

Key architectural rule: Supabase is used strictly as managed Postgres + file storage. Flask connects using the `service_role` key (or a direct Postgres connection string) and owns 100% of authentication, sessions, and business logic. Supabase Auth, Row Level Security policies, and the auto-generated PostgREST API are not used — this avoids two competing identity systems and the "silent empty query result" trap RLS creates for non-Supabase-Auth clients.

4. System Architecture

4.1 High-Level Diagram

4.2 Request Lifecycle Example — Teacher Submitting a Roll Call

1. Teacher logs in (Flask-Login session cookie set).
2. Teacher opens "Today's Sessions" — Flask queries `timetable_sessions` joined against today's date and the teacher's ID.
3. Teacher selects a session → Flask renders the roster (all students in `student_subject_enrollment` for that class + subject).
4. Teacher marks present/absent for each student, submits the form (CSRF-protected POST).
5. Flask writes rows to `attendance_records` and marks the session `submitted = true`, `submitted_at = now()`.
6. Any later edit to that roll call writes a row to `attendance_audit_log` instead of silently overwriting.

4.3 Why Server-Rendered, Not a Separate API + SPA

A separate JS frontend calling a Flask JSON API would add CORS handling, token-based auth (JWT storage, refresh flows), and a second deployment target — with no real benefit for a role-based internal school tool. Server-rendered Jinja2 keeps auth simple (one cookie-based session), reduces the number of moving parts your team has to operate, and is faster to build for a small team. Chart data can still be fetched asynchronously as JSON for the dashboard charts specifically, without needing a full SPA architecture.

5. Roles & Permissions Matrix

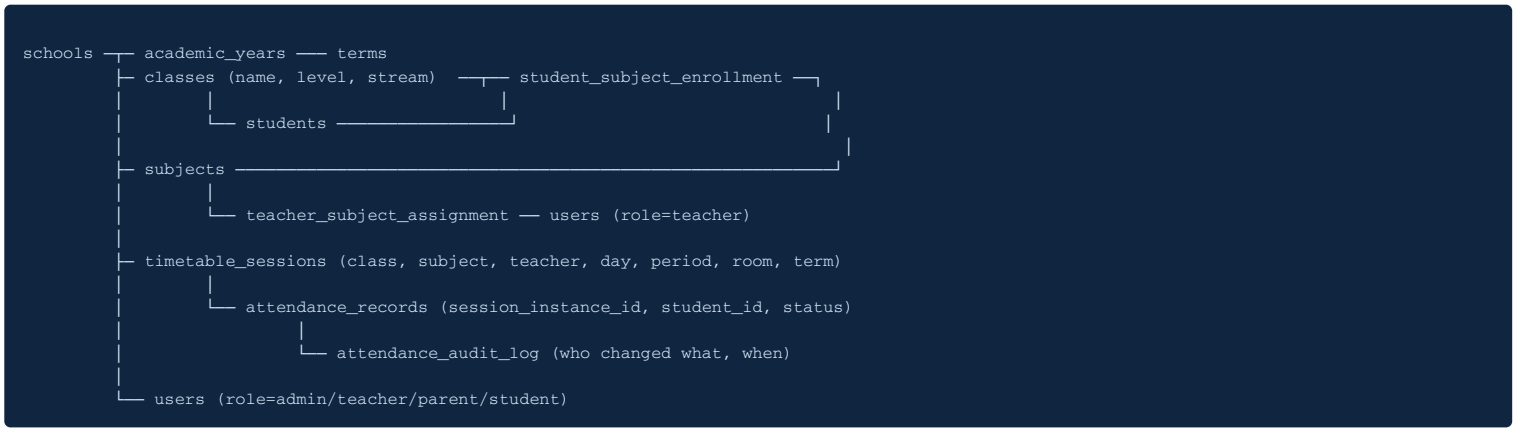
Role	Created By	Login Method	Key Permissions
Super Admin (DOS)	Seeded once at deployment via CLI script	Email + password	Full access: manage users, classes, subjects, timetable, enrollment, all dashboards, all student profiles, audit logs
Admin Staff (e.g. secretary / front office)	Super Admin	Email + password	Search & view students/classes, register new students, no config/timetable access
Teacher	Super Admin	Email/phone + password (forced reset on first login)	View own timetable sessions only, submit/edit own roll calls, view own subject's attendance stats for classes they teach
Class Teacher (optional, form-teacher role)	Super Admin (flag on a Teacher account)	Same as Teacher	All Teacher permissions + full attendance view (all subjects) for their assigned class only
Parent (Phase 2)	Self-registers using an access code issued at student registration	Phone number + access code	View own child's profile and attendance only
Student (Phase 2, optional)	Bulk-imported by Admin	Student ID + PIN	View own attendance only

Accountability by design: No role above self-registers except Parent (Phase 2), and even that is scoped to a code tied to one specific student. Every account is traceable to who created it and when — this closes the loophole of a teacher creating a second account to bypass tracking.

6. Database Schema (Full Design)

Designed relationally, with the elective/stream scalability requirement built in from day one — S1/S2 simply auto-enrolls every student into every subject their class offers; S3–S6 uses the exact same tables with real per-student elective selection, with no schema change required later.

6.1 Entity Relationship Overview



6.2 Core Tables

schools

```
id (PK)
name
address
created_at
```

Included even for a single-school deployment — costs nothing now, prevents a painful migration if the system is ever deployed to a second school.

academic_years / terms

```
academic_years: id (PK), school_id (FK), label (e.g. "2026"), start_date, end_date

terms: id (PK), academic_year_id (FK), name (e.g. "Term 1"), start_date, end_date, is_current (bool)
```

Every attendance session and enrollment is scoped to a term. Without this, weekly/termly charts break the moment data crosses a term boundary.

classes

```
id (PK)
school_id (FK)
level        -- e.g. "S1", "S2", "S3"
stream       -- e.g. "Blue", "Red"  (nullable if no streaming)
academic_year_id (FK)
class_teacher_id (FK -> users, nullable)
```

subjects

```
id (PK)
school_id (FK)
name          -- e.g. "Mathematics"
is_elective   -- bool, false for all S1/S2 subjects
```

student_subject_enrollment

```
id (PK)
student_id (FK)
subject_id (FK)
term_id (FK)
```

This is the table that makes the system scale. For S1/S2: a script auto-inserts a row for every student × every subject offered by their class at term start. For S3+: this becomes a real per-student elective selection step in the admin UI. The attendance and session logic below never needs to know the difference.

users

```
id (PK)
```

```
school_id (FK)
role            -- enum: super_admin, admin_staff, teacher, parent, student
full_name
email
phone
password_hash
must_reset_password  -- bool, true on first login for staff-provisioned accounts
is_active
created_by (FK -> users, nullable)
created_at
```

teacher_subject_assignment

```
id (PK)
teacher_id (FK -> users)
subject_id (FK)
class_id (FK)
term_id (FK)
```

timetable_sessions (the timetable "template")

```
id (PK)
class_id (FK)
subject_id (FK)
teacher_id (FK -> users)
term_id (FK)
day_of_week     -- Mon..Fri
period_number   -- 1..8
start_time
end_time
room
```

session_instances (one concrete, dated occurrence of a timetable_session)

```
id (PK)
timetable_session_id (FK)
session_date
status              -- scheduled | rollcall_submitted | flagged_missed | cancelled
submitted_by (FK -> users, nullable)
submitted_at
grace_deadline      -- session end_time + grace period (e.g. +20 min)
```

Generated automatically each day (or in a rolling batch) from `timetable_sessions`. This table is the backbone of the missed-rollcall flagging logic (see Section 10.3).

attendance_records

```
id (PK)
session_instance_id (FK)
student_id (FK)
status          -- present | absent | late | excused
marked_by (FK -> users)
marked_at
```

attendance_audit_log

```
id (PK)
attendance_record_id (FK)
changed_by (FK -> users)
old_status
new_status
changed_at
reason (nullable, free text)
```

Every edit to an already-submitted attendance record writes here instead of overwriting silently. This is non-negotiable for the accountability goal — without it, a teacher could submit a roll call, then quietly edit it later to cover a missed lesson.

students

```
id (PK)
school_id (FK)
class_id (FK)
student_number       -- unique, school-assigned ID (not name-based)
full_name
date_of_birth
gender
guardian_name
guardian_phone
photo_storage_path   -- private Supabase Storage path, not a public URL
enrollment_date
is_active
```

```
parental_consent_on_file   -- bool, required before photo upload is used
```

parent_access (Phase 2 — not built in v1, table reserved)

```
id (PK)
student_id (FK)
access_code (hashed)
phone
issued_at
used_at (nullable)
```

6.3 Indexing Notes

- Composite index on `session_instances(session_date, status)` — the auto-flagging job scans this daily.
- Composite index on `attendance_records(student_id, session_instance_id)` — powers the student profile page.
- Index on `students(class_id)` and `students(student_number)` — powers the class → student search flow.

7. Complete Sitemap / Page Map by Role

7.1 Public / Shared

/login — single login page for all roles; redirects by role after auth.

/logout — clears session.

/reset-password/<token> — password reset via emailed link.

7.2 Super Admin / DOS

/admin/dashboard — school-wide overview: attendance %, top-flagged students, top-flagged teachers.

/admin/users → **/admin/users/new** — manage & provision teacher/staff accounts.

/admin/classes → **/admin/classes/new** — manage classes & streams.

/admin/subjects — manage subject list, mark elective vs. compulsory.

/admin/timetable — visual grid builder mapping class × day × period → subject + teacher.

/admin/students → **/admin/students/new** — bulk/individual student registration incl. photo upload.

/admin/students/<id> — full student profile (shared with parent-lookup flow, see 7.4).

/admin/attendance/flags — all currently flagged missed sessions (by teacher, by class).

/admin/reports — generate/download term reports (PDF/CSV export).

/admin/audit-log — full history of attendance edits.

7.3 Teacher

/teacher/dashboard — today's sessions, pending roll calls, own flagged-missed history.

/teacher/sessions/<session_instance_id>/rollcall — the core roll call form (mobile-first).

/teacher/my-classes — attendance stats for classes/subjects this teacher teaches.

7.4 Front Office / Parent Lookup Flow (staff-assisted, Phase 1)

/lookup/classes — list of classes (searchable).

/lookup/classes/<class_id>/students — student roster with photos for that class.

/lookup/students/<id> — same student profile page as 7.2, read-only for front-office role.

7.5 Shared Component: Dashboard Views

/dashboard/attendance?level=lesson|daily|weekly|monthly|termly&scope=class|subject|student|teacher — a single parameterised dashboard endpoint feeding Chart.js visualisations, reused across admin and teacher views with permission scoping applied server-side.

8. Page-by-Page Design Detail

8.1 Login Page

Purpose: single entry point for all roles.
Elements: email/phone field, password field, "forgot password" link, school logo/branding.
Logic: on success, redirect based on `role` — super_admin/admin_staff → `/admin/dashboard`, teacher → `/teacher/dashboard`. If `must_reset_password` is true, force redirect to a password-set page before anything else.

8.2 Admin Dashboard

Purpose: the DOS's command centre — the single page that answers "where is the problem, right now."

- **Top row:** four summary cards — overall attendance % today, sessions flagged missed today, students currently below an attendance threshold, teachers with repeated flags this term.
- **Chart block 1:** attendance trend line, toggle between daily / weekly / monthly / termly.
- **Chart block 2:** bar chart of attendance % by class, worst-performing highlighted in red.
- **Chart block 3:** pie chart of flagged-missed sessions by teacher, for quick accountability review.
- **Table:** "Students needing attention" — auto-sorted list of students below the attendance threshold, each row clickable through to their profile.

8.3 Timetable Builder (Admin)

Purpose: the foundation the entire system depends on.
Layout: a grid — rows = periods, columns = days, one grid per class. Each cell is an editable dropdown pair: subject + teacher. Saving a cell writes/updates a row in `timetable_sessions`.
Validation: prevent double-booking a teacher into two classes at the same day/period; warn (don't block) if a subject is scheduled fewer times per week than its expected minimum.
See Section 9 for the full capture workflow — including bulk import from an existing Excel timetable, per-term versioning, and how a mid-term change is handled.

8.4 Student Registration (Admin / Front Office)

Purpose: capture full student information, including photo, for S1/S2 first.
Fields: full name, date of birth, gender, class/stream, guardian name & phone, student number (auto-generated, editable), photo upload (resized/compressed client-side before upload where possible), parental consent checkbox (mandatory — form will not submit without it).
Logic on submit: creates the `students` row, uploads photo to the private Supabase Storage bucket, then auto-runs the enrollment script inserting rows into `student_subject_enrollment` for every subject the student's class offers (S1/S2 case). Redirects to the new student's profile page.

8.5 Student Profile Page **(shared: Admin view is editable, Front-Office/Parent-lookup view is read-only)**

- **Header:** photo, name, student number, class/stream, guardian contact.
- **Attendance summary card:** overall attendance % this term, trend arrow vs. last term.
- **Subject breakdown table:** per-subject attendance %, worst subject highlighted.
- **Timeline/calendar view:** visual grid of the term, each day/session colour-coded present/absent/flagged.
- **Recent absences list:** most recent missed sessions, with subject and teacher shown.

This single page is what a staff member pulls up when a parent asks "why did my child fail" — it is designed to answer that question visually within seconds.

8.6 Teacher Dashboard

- **Today's sessions list:** each session shown as a card — class, subject, time, room, and a clear status badge (Not yet submitted / Submitted / Flagged Missed).
- **Quick action:** tapping an unsubmitted session goes straight into the roll call form — minimal navigation, since this is the highest-friction, most time-sensitive action in the whole system.
- **Own accountability panel:** a small, honest summary — "You have missed 2 roll calls this term" — visible only to that teacher and the DOS, not other teachers.

8.7 Roll Call Form (the most critical UI in the system)

Design priorities: must be fast on a phone, large tap targets, minimal scrolling for a class of 40+.

- Every student defaults to **Present** (a teacher taps only the exceptions — absent/late — which is faster than tapping every present student individually).
- Each row: student photo thumbnail + name + a 3-way toggle (Present / Absent / Late).
- Search/filter box at top for quickly finding a specific student in a large class.
- Single "Submit Roll Call" button at the bottom, sticky/fixed so it's always reachable without scrolling back up.
- On submit: confirmation state shown clearly; if the network fails, the form retains entered data and shows a retry option rather than silently losing the input.

8.8 Attendance Flags Page (Admin)

Two tabs: **Missed Sessions** (session, class, subject, teacher, date, time since flagged) and **Students at Risk** (student, class, attendance %, subjects most affected). Both sortable and filterable by class, subject, date range, and term.

8.9 Reports Page

Generate a PDF/CSV export scoped by class, term, or individual student — for printed parent meetings or record-keeping. Not required for MVP demo but the endpoint should exist as a stub from early on so it isn't a late scope surprise.

9. Timetable Capture: Full Data-Entry Workflow

This section fills the gap between "the timetable exists as a table in the database" (Section 6) and "the timetable is on someone's desk as an Excel sheet or a printed grid." It covers exactly how that real-world timetable gets into the system, who does it, and how it is kept correct as terms change and disruptions happen.

9.1 Two Entry Methods, Offered Together

Every school already has a timetable in some form before this system exists — almost always an Excel sheet or a printed master grid produced by the DOS at the start of term. Forcing that to be re-typed cell-by-cell into a web grid is slow and error-prone, so the system supports two entry paths into the same underlying `timetable_sessions` table:

Method	Best for	How it works
Bulk Import (Excel/CSV upload)	Initial term setup — getting the whole existing timetable in at once	DOS uploads a spreadsheet in a defined column format; the system parses, validates, and previews it before committing anything
Manual Grid Entry	Corrections, small schools without a digital timetable yet, and all day-to-day edits after initial setup	The visual grid described in Section 8.3 — one cell at a time, immediate validation

9.2 Bulk Import Workflow (Step-by-Step)

1. DOS downloads a **template spreadsheet** from `/admin/timetable/import` — columns: `class, stream, day_of_week, period_number, start_time, end_time, subject, teacher_email, room`.

2. DOS fills the template from the school's existing timetable (copy-paste from the current Excel sheet is normally fast, since the column shape mirrors how most school timetables are already laid out) and uploads the file.

3. Flask parses the file (using a library such as `openpyxl` or `pandas`) row by row and runs it through the **same validation rules** as manual entry (Section 9.4, below) — nothing is written to the database yet.

4. The system shows a **preview screen**: a rendered grid of what will be created, with every row that failed validation clearly listed separately (e.g. "Row 14: teacher 'j.mukasa@school.ug' not found — check spelling or create the account first", "Row 22: teacher already booked for S2 Red at this day/period").

5. DOS fixes errors either in the spreadsheet and re-uploads, or corrects individual rows inline on the preview screen.

6. On confirmation, the system commits all valid rows as `timetable_sessions` records in a single transaction — either the whole import succeeds, or none of it does, so a class is never left half-configured.

7. Session-instance generation (Section 10.2) then picks up the newly created timetable automatically from the next scheduled run.

9.3 Manual Grid Entry Workflow

1. DOS opens `/admin/timetable`, selects a class from a dropdown — the grid loads showing that class's current timetable (empty on first use).

2. DOS clicks an empty cell (a day/period intersection) → a small inline form appears: subject dropdown, teacher dropdown (filtered to teachers already linked to that subject via `teacher_subject_assignment`, with an option to assign a new teacher-subject link on the fly if needed), room field.

3. On save, the system validates immediately (Section 9.4, below) and either writes the cell or shows an inline error without losing the rest of the grid's state.

4. Filled cells display as a compact card showing subject + teacher initials; clicking an existing cell allows edit or delete.

5. A "Copy to another class" action lets the DOS duplicate a similar structure (useful for classes with near-identical timetables) and then adjust the differences, rather than starting from a blank grid every time.

9.4 Validation Rules (Apply Identically to Both Entry Methods)

Rule	Behaviour
Teacher double-booking	**Blocking error** — a teacher cannot be assigned to two different classes at the same day + period
Room double-booking	**Warning** (not blocking) — flagged so DOS can decide if it's intentional (e.g. shared lab time) or a mistake
Teacher not linked to subject	**Blocking error** unless DOS explicitly confirms creating the link on the spot
Subject below expected weekly frequency	**Warning** — e.g. a subject configured to need 5 periods/week but only 3 are scheduled
Overlapping time ranges within the same class	**Blocking error** — a class cannot have two subjects in the same period

9.5 Per-Term Versioning — Why the Timetable Cannot Just Be "Edited"

Timetables change every term (sometimes mid-term). Because every `timetable_sessions` row is scoped to a `term_id` (Section 6.2), starting a new term does not overwrite the old timetable — it creates a new, independent set of rows. This matters for two reasons: first, historical attendance reports for Term 1 must continue to reflect Term 1's actual timetable even after Term 2 begins; second, it lets the DOS **prepare next term's timetable in advance** while the current term is still live, without any risk of the two colliding.

> **New term setup:** DOS selects "Start New Term" → chooses to either build the new timetable from scratch (bulk import or manual grid) or **clone the previous term's timetable** as a starting point and edit only what changed (the common case — most of a school's timetable is stable term to term, only a few teacher or subject changes occur).

9.6 Mid-Term Changes (Substitute Teachers, Room Changes, Schedule Shifts)

Two distinct situations are handled differently, deliberately:

- **Permanent change** (e.g. a teacher leaves, a subject moves to a new period for the rest of term) → DOS edits the `timetable_sessions` row directly through the grid. This affects all *future* session-instance generation from that point forward; past session_instances (and their attendance history) are untouched, since they were already generated as independent records.
- **One-off change** (e.g. today only, a substitute covers a single lesson because the regular teacher is sick) → DOS uses a lightweight **"Reassign Today's Session"** action directly on that specific `session_instance`, which overrides just the `teacher_id` for that one instance without touching the underlying timetable template at all. This keeps one-off disruptions from requiring a full timetable edit and re-edit-back cycle.

9.7 Who Is Allowed to Capture the Timetable

Only **Super Admin (DOS)** has access to `/admin/timetable` and the import endpoint (see the Roles & Permissions Matrix, Section 5). Teachers cannot self-edit their own timetable slots — this is intentional: the timetable is the accountability baseline against which missed roll calls are measured, so it must stay under the DOS's control, with every change attributable to them via the same audit principle used for attendance edits (Section 6.2, `attendance_audit_log` pattern — a parallel `timetable_audit_log` table is a reasonable addition if timetable-tampering disputes ever arise in practice; not required for v1 but cheap to add if needed).

10. Core Logic Flows (Step-by-Step)

9.1 Teacher Account Provisioning

1. DOS opens `/admin/users/new`, enters teacher's name, email/phone, assigns subjects + classes via `teacher_subject_assignment`.

2. System generates a random temporary password, creates the `users` row with `must_reset_password = true`.

3. System emails/SMS's the temporary credentials to the teacher (or DOS hands them over directly if no email infra yet).

4. Teacher's first login forces a password-set page before any other page is reachable.

9.2 Timetable → Session Instance Generation

1. A scheduled job (daily, e.g. 05:00) or an on-demand admin action reads all `timetable_sessions` rows for the current term.

2. For each row whose `day_of_week` matches today, the job creates a `session_instances` row for today's date, status `scheduled`, with `grace_deadline = end_time + 20 minutes` (configurable).

3. This decouples the "template" timetable from the day-to-day operational record — the template can change term to term without touching historical instances.

9.3 Missed Roll Call Auto-Flagging

1. A second scheduled job runs periodically (e.g. every 15 minutes) throughout the school day.

2. It selects all `session_instances` where `status = 'scheduled'` and `now() > grace_deadline`.

3. Each matching row is updated to `status = 'flagged_missed'`.

4. This immediately affects the DOS dashboard (flag count) and the teacher's own accountability panel — no manual step required.

9.4 Roll Call Submission

1. Teacher opens a `scheduled` session_instance → system pulls roster from `student_subject_enrollment` for that class + subject + term.

2. Teacher marks each student (defaulting to Present), submits.

3. Flask writes one `attendance_records` row per student and sets `session_instances.status = 'rollcall_submitted'`, `submitted_by`, `submitted_at`.

4. If the session was already `flagged_missed` before a late submission, the flag remains visible in historical reporting (it happened late) even though current status updates — the system does not erase the fact that it was late.

9.5 Editing an Already-Submitted Roll Call

1. Teacher (or DOS override) opens a submitted session and changes a student's status.

2. Before updating `attendance_records`, the system writes the old value, new value, who, and when into `attendance_audit_log`.

3. The DOS audit-log page surfaces any edits made more than a set time after original submission for review — a pattern of late edits is itself a signal worth the DOS's attention.

9.6 Student Registration & Auto-Enrollment (S1/S2 case)

1. Admin submits the registration form (Section 8.4), including photo and signed consent confirmation.

2. System creates the `students` row and uploads the photo to private storage.

3. System looks up every subject associated with the student's class for the current term and bulk-inserts `student_subject_enrollment` rows — this is what makes the student appear automatically on every relevant teacher's roster from day one.

9.7 Parent / Front-Office Lookup Flow

1. Staff member opens `/lookup/classes`, selects the parent's child's class.

2. Staff selects the student from the photo roster (photo is the key identification aid for new/unfamiliar S1/S2 faces).

3. System renders the read-only student profile (Section 8.5) — staff talks the parent through it directly.

9.8 Dashboard Aggregation Query Pattern

All dashboard charts are powered by one parameterised query pattern: aggregate `attendance_records` joined through `session_instances` → `timetable_sessions`, grouped by the requested dimension (`student_id`, `class_id`, `subject_id`, or `teacher_id`) and the requested time bucket (day/week/month/term via `session_date` and `term_id`). This single reusable query shape, exposed as a JSON endpoint, is what feeds every Chart.js visualisation across every dashboard — built once, reused everywhere, rather than a bespoke query per chart.

11. Security, Privacy & Compliance

10.1 Authentication & Session Security

- Passwords hashed with Werkzeug's `generate_password_hash` (bcrypt/scrypt-based) — never stored plain.
- Session cookies: `SESSION_COOKIE_SECURE=True`, `HTTPONLY=True`, reasonable timeout with re-auth for sensitive admin actions.
- CSRF protection on every form via Flask-WTF.
- No self-signup route exists anywhere in the app for staff roles — provisioning is admin-only, by design.

10.2 Data Access Control

- Teachers can only query sessions/students where they are the assigned teacher (enforced server-side in every query, not just hidden in the UI).
- Student photos are stored in a **private** Supabase Storage bucket; pages render them via short-lived signed URLs generated by Flask, never a permanently public link.

10.3 Minors' Data & Legal Basis

- Written parental/guardian consent is captured at registration before a photo is used in the system (a mandatory checkbox tied to a signed physical or digital consent record).
- Consent language should disclose that data is stored on international cloud servers (Supabase's available regions do not include East Africa), relevant under Uganda's Data Protection and Privacy Act, 2019.
- A data retention policy should be defined (e.g. how long a student's records and photo are kept after they leave the school) before launch.

10.4 Audit & Accountability

- Every attendance edit is logged immutably (Section 6.2, `attendance_audit_log`).
- Every user account records who created it and when.

10.5 Database Access Hygiene

- Flask connects with the Supabase `service_role` key or a direct DB role — never exposed to any client-side code.
- Row Level Security left off (or explicitly bypassed via service role) since Flask is the sole gatekeeper — documented clearly so a future contributor doesn't "fix" this by turning RLS on and breaking every query.

12. Environments, Deployment & DevOps

11.1 Three Environments

Environment	Database	App Host	Purpose
Local Development	SQLite or a free local-only Supabase dev project	Flask dev server (`flask run`)	Day-to-day coding, fast iteration
Staging	Separate Supabase project ("staging")	Render/Railway staging service	Test migrations and full flows with realistic data before touching production
Production	Separate Supabase project ("production")	Render/Railway production service	Live school data

11.2 Migration Discipline

- All schema changes go through Alembic migration files — never hand-edited via the Supabase dashboard table editor.
- Workflow: write model change → `alembic revision --autogenerate` → review the generated migration → apply to local → apply to staging → verify → apply to production.

11.3 Connection Pooling

Application connects via Supabase's pooled connection string (transaction mode, port `6543`), not the direct connection (port 5432), to stay within the free tier's connection limits under concurrent teacher usage during peak roll-call periods.

11.4 Scheduled Jobs

Session-instance generation (10.2) and missed-rollcall flagging (10.3) run as scheduled jobs. Two implementation options: **APScheduler** running inside the Flask process (simplest, works on Render/Railway's persistent processes), or the host's native cron/scheduled-job feature calling a dedicated Flask CLI command. The native host cron option is more resilient to app restarts and is the recommended default.

11.5 Backups

A weekly scheduled job runs `pg_dump` against production and stores the dump in a private location (e.g. a private GitHub repo used purely for backups, or Backblaze B2 free tier) — required because Supabase's free tier does not include automated backups.

11.6 Keeping the Free Tier Project Alive

A lightweight scheduled ping (a simple authenticated query run every few days) prevents Supabase's free-tier auto-pause during low-activity periods such as school holidays.

13. Project Folder Structure

```
school_attendance_system/
│
├── app/
│   ├── __init__.py            # app factory, extension init
│   ├── config.py               # env-based config classes (Dev/Staging/Prod)
│   ├── extensions.py           # db, login_manager, csrf, mail instances
│   │
│   ├── models/
│   │   ├── school.py
│   │   ├── user.py
│   │   ├── student.py
│   │   ├── class_.py
│   │   ├── subject.py
│   │   ├── enrollment.py
│   │   ├── timetable.py
│   │   ├── session_instance.py
│   │   ├── attendance.py
│   │   └── audit_log.py
│   │
│   ├── blueprints/
│   │   ├── auth/
│   │   ├── admin/
│   │   │   ├── users.py
│   │   │   ├── classes.py
│   │   │   ├── subjects.py
│   │   │   ├── timetable.py
│   │   │   ├── students.py
│   │   │   ├── flags.py
│   │   │   └── reports.py
│   │   ├── teacher/
│   │   │   ├── dashboard.py
│   │   │   └── rollcall.py
│   │   ├── lookup/             # front-office / parent-assisted lookup
│   │   └── dashboard/          # shared parameterised chart-data endpoints
│   │
│   ├── jobs/
│   │   ├── generate_sessions.py
│   │   └── flag_missed_rollcalls.py
│   │
│   ├── services/
│   │   ├── enrollment_service.py
│   │   ├── storage_service.py   # signed URL generation, photo upload
│   │   └── audit_service.py
│   │
│   ├── templates/
│   │   ├── base.html
│   │   ├── auth/
│   │   ├── admin/
│   │   ├── teacher/
│   │   └── shared/
│   │
│   └── static/
│       ├── css/
│       └── js/
│
├── migrations/                 # Alembic
├── tests/
├── scripts/
│   ├── seed_admin.py            # creates the first super_admin account
│   └── backup_db.py
├── requirements.txt
├── .env.example
└── README.md
```

14. Implementation Roadmap (Phase by Phase)

Structured so each phase produces something runnable and testable — nothing is built in an un-verifiable block.

Phase 0 — Project Setup (Week 1)

- Create GitHub repo, Flask app factory skeleton, `.env` convention.
- Create three Supabase projects: dev, staging, production.
- Set up SQLAlchemy + Alembic, confirm a migration can run against all three environments.
- Deploy a "hello world" Flask app to Render/Railway to confirm the deployment pipeline works end-to-end before any real feature exists.

Phase 1 — Core Data Model & Auth (Weeks 2–3)

- Build all models from Section 6; generate and apply the first Alembic migration.
- Build `scripts/seed_admin.py` to create the first super_admin.
- Build login/logout, Flask-Login integration, forced password reset flow.
- Build Admin → Users page (provision teacher accounts).
- **Checkpoint:** DOS account can log in and create a teacher account; teacher can log in and reset password.

Phase 2 — School Structure Setup (Weeks 3–4)

- Build Classes, Subjects, and Student registration (with photo upload to private Supabase Storage) pages.
- Build the enrollment service (auto-enroll S1/S2 students into all class subjects).
- **Checkpoint:** a full class of test students can be registered with photos and appear correctly enrolled in every subject.

Phase 3 — Timetable & Session Engine (Weeks 4–5)

- Build the Timetable Builder grid UI and `timetable_sessions` CRUD.
- Build the session-instance generation job (10.2) and wire it to the host's scheduler.
- **Checkpoint:** setting up a timetable for one class produces correct daily session_instances automatically.

Phase 4 — Roll Call Core (Weeks 5–7)

- Build the Teacher Dashboard (today's sessions list).
- Build the mobile-first Roll Call form and submission logic (10.4).
- Build the audit log and edit flow (10.5).
- Build the missed-rollcall auto-flagging job (10.3).
- **Checkpoint:** a teacher can submit a roll call end-to-end on a phone; an unsubmitted session correctly flags itself after the grace period.

Phase 5 — Dashboards & Insights (Weeks 7–9)

- Build the shared parameterised aggregation endpoint (10.8).
- Build the Admin Dashboard (Section 8.2) with Chart.js visualisations at lesson/daily/weekly/monthly/termly levels.
- Build the Attendance Flags page (8.8) and Students-at-Risk highlighting.
- **Checkpoint:** DOS can see, at a glance, which students and teachers need attention, across all time views.

Phase 6 — Student Profile & Lookup Flow (Weeks 9–10)

- Build the Student Profile page (8.5) with attendance timeline/calendar view.
- Build the class → student search / lookup flow (7.4) for front-office/parent-assisted use.
- **Checkpoint:** staff can find any student by class and show a parent a complete, accurate attendance picture in under a minute.

Phase 7 — Hardening (Weeks 10–11)

- CSRF, session security, signed-URL photo access review (Section 10).
- Set up scheduled backups and the Supabase keep-alive ping.
- Load-test roll call submission with simulated concurrent teachers (peak-period scenario).
- Full run-through of the testing checklist (Section 14).

Phase 8 — Pilot (Weeks 12–14)

- Deploy to production Supabase + Render/Railway.

- Onboard one S1 and one S2 class fully (real students, real photos, real consent).
- Run the pilot rollout plan (Section 15); collect teacher feedback specifically on the roll call form.

Phase 9 — Full Rollout (Week 15+)

- Fix issues surfaced in the pilot.
- Onboard remaining S1/S2 classes.
- Train all teachers and front-office staff.
- Go live school-wide, with a documented paper-based fallback procedure for any system downtime.

15. Testing & QA Checklist

Area	What to Verify
Auth	No route is reachable without login except `/login`; role-based redirects work; forced password reset cannot be bypassed by direct URL
Permissions	A teacher cannot view/submit roll call for a session that is not theirs, even via direct URL manipulation
Enrollment	Registering a new S1/S2 student correctly enrolls them into every subject for their class
Session generation	Timetable changes correctly reflect in the next day's generated session instances, without duplicating past instances
Roll call submission	Submitting marks all students correctly; defaults to present; partial submissions are not silently lost
Auto-flagging	A session with no submission is flagged exactly once, at the correct grace-deadline time
Audit log	Editing a submitted roll call always produces a log entry; no code path allows a silent overwrite
Dashboards	Chart numbers match a manual query against the same date range, for at least one class as a spot-check
Photo privacy	Photo URLs expire; a stale signed URL cannot be reused after expiry
Mobile UI	Roll call form is usable one-handed on a mid-range Android phone with a slow connection
Backups	A restored backup produces a working, consistent database

16. Pilot Rollout Plan

1. **Select pilot group:** one S1 class and one S2 class (two streams), to keep the dataset small and feedback fast.
2. **Full registration event:** a single dedicated session to register all pilot students with photos and signed parental consent.
3. **Timetable entry:** DOS enters the real timetable for just these two classes.
4. **Teacher training session:** 30–45 minutes, hands-on, focused entirely on the roll call form — this is the page teachers will touch daily, so friction here determines adoption.
5. **Two-to-four week live pilot** with daily roll call use, DOS monitoring the dashboard and flags page actively.
6. **Feedback checkpoint:** structured feedback from pilot teachers specifically on speed and usability of the roll call form, and from DOS on whether the dashboard actually answers "who needs attention."
7. **Fix and adjust** before expanding to the remaining S1/S2 classes.
8. **Define the fallback procedure** (paper roll call sheet, entered into the system by end of day) for any period of system downtime — communicated to all teachers before full rollout, not improvised later.

17. Appendix: Environment Variables & Config

```
FLASK_ENV=production
SECRET_KEY=<random, never committed>

# Supabase (per environment: dev / staging / production)
DATABASE_URL=postgresql://...:6543/postgres   # pooled connection, port 6543
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_SERVICE_ROLE_KEY=<server-side only, never exposed to client>
SUPABASE_STORAGE_BUCKET=student-photos

# Mail (password resets)
MAIL_SERVER=smtp-relay.brevo.com
MAIL_PORT=587
MAIL_USERNAME=<...>
MAIL_PASSWORD=<...>

# Scheduler
GRACE_PERIOD_MINUTES=20
BACKUP_DESTINATION=<private storage location>
```

> **Never commit** `.env` to version control. Use `.env.example` with placeholder values in the repo, and configure real values directly in Render/Railway's environment variable dashboard per environment.

Quick Reference — Section Map

If you need to...	Go to
Remember why a tech choice was made	Section 3
Add a new table or change the schema	Section 6, then follow migration discipline in 12.2
Build a new page	Check Section 7 (sitemap) then Section 8 (page detail) for the pattern to follow
Capture or update the timetable	Section 9 — covers bulk import, manual entry, versioning, and mid-term changes
Understand how a background process works	Section 10
Check a security requirement before shipping	Section 10
Know what to build next	Section 13 — follow the phase order

— End of design document. This plan is intended as a living reference: update it as decisions evolve during implementation, particularly the schema (Section 6) and roadmap (Section 13), so it stays the single source of truth for the build.